PINN Example: Solving the 1D Heat Equation

Cyrus Young

April 21, 2026

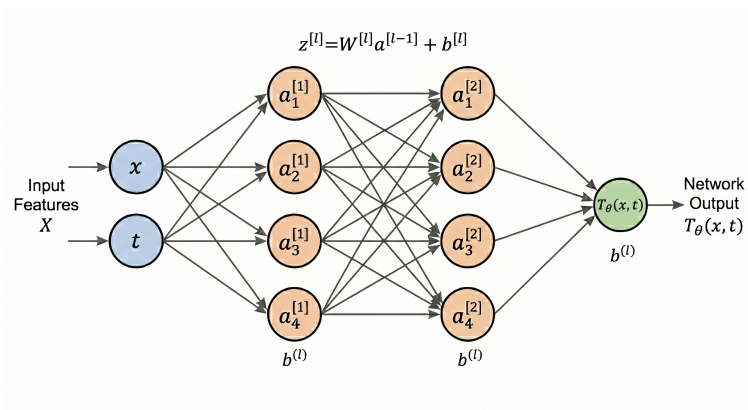

Figure 1: PINN overview for the 1D heat equation.

0.1 How would one solve the 1D heat equation with a PINN?

To make the PINN methodology concrete, consider the one-dimensional heat equation

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} + q(x,t),$$

defined on a spatial domain $x \in [0, L]$ and for $t > 0$, where $\alpha > 0$ is the thermal diffusivity and $q(x,t)$ is a prescribed heat source. The problem is closed by an initial condition

$$T(x,0) = f(x)$$

and Dirichlet boundary conditions

$$T(0,t) = g_0(t), \qquad T(L,t) = g_L(t).$$

The goal is to find a function $T(x,t)$ satisfying all three of these requirements simultaneously.

Big picture: the implementation loop is $(x,t) \mapsto T_\theta(x,t)$ via a forward pass, automatic differentiation to obtain T_t and T_{xx}, evaluation of the PDE residual and the total loss, gradients of that loss with respect to the weights and biases, and a parameter update that produces the next set of predictions.

The neural network as a function approximator

A PINN replaces the unknown solution $T(x,t)$ with the output of a feedforward neural network $T_\theta(x,t)$. The network takes a single space-time point as its two-dimensional input and produces a scalar temperature prediction:

$$a^{[0]} = \begin{bmatrix} x \\ t \end{bmatrix}, \qquad a^{[N]} = T_\theta(x,t).$$

The forward pass through the network proceeds layer by layer. For $\ell = 1, \ldots, N$, a pre-activation vector is formed by a linear transformation of the previous layer's output,

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]},$$

and a nonlinear activation function σ is then applied elementwise to produce the next layer's output,

$$a^{[\ell]} = \sigma\left(z^{[\ell]}\right), \qquad \ell = 1, \ldots, N-1.$$

Here $W^{[\ell]}$ is the weight matrix for layer ℓ, $b^{[\ell]}$ is the corresponding bias vector, $z^{[\ell]}$ is the pre-activation at layer ℓ, and $a^{[\ell]}$ is its post-activation output. The final layer $a^{[N]}$ is a single scalar and is interpreted directly as the predicted temperature $T_\theta(x,t)$; no activation function is applied at the output. The trainable parameters are $W^{[\ell]}$ and $b^{[\ell]}$ for $\ell = 1, \ldots, N$; the quantities $z^{[\ell]}$, $a^{[\ell]}$, T_θ, T_t, and T_{xx} are intermediate values computed from the current parameters during each forward pass.

Encoding the physics via automatic differentiation

The network itself knows nothing about the heat equation. Physics enters through the loss function. Because $T_\theta(x,t)$ is a differentiable composition of smooth functions, its partial derivatives with respect to the inputs can be computed exactly by automatic differentiation:

$$T_t = \frac{\partial T_\theta}{\partial t}, \qquad T_{xx} = \frac{\partial^2 T_\theta}{\partial x^2}.$$

Substituting the network output into the heat equation yields the PDE residual

$$R_\theta(x,t) = \frac{\partial T_\theta}{\partial t} - \alpha \frac{\partial^2 T_\theta}{\partial x^2} - q(x,t).$$

If T_θ were the true solution, this residual would vanish everywhere in the domain. Training is designed to make it as small as possible. Automatic differentiation is essential because the residual requires exact derivatives of the network output with respect to its inputs at many points, which would be cumbersome and error-prone to compute by hand or by finite differences.

The loss function

The training objective is a weighted sum of three mean-squared loss terms, one for each requirement imposed on the solution.

The *PDE residual loss* penalises violations of the heat equation at a set of N_f interior collocation points, i.e., sampled space-time points where the PDE is enforced, $\{(x_i^{(f)}, t_i^{(f)})\}$:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| R_\theta(x_i^{(f)}, t_i^{(f)}) \right|^2.$$

The *initial-condition loss* enforces the prescribed initial temperature profile $f(x)$ at N_0 spatial points sampled at $t = 0$:

$$\mathcal{L}_{\text{IC}} = \frac{1}{N_0} \sum_{i=1}^{N_0} \left| T_\theta(x_i^{(0)}, 0) - f(x_i^{(0)}) \right|^2.$$

The *boundary-condition loss* enforces the prescribed boundary temperatures g_0 and g_L at both ends of the domain, sampled at N_b time points:

$$\mathcal{L}_{\text{BC}} = \frac{1}{N_b} \sum_{i=1}^{N_b} \left| T_\theta(0, t_i^{(b)}) - g_0(t_i^{(b)}) \right|^2 + \frac{1}{N_b} \sum_{i=1}^{N_b} \left| T_\theta(L, t_i^{(b)}) - g_L(t_i^{(b)}) \right|^2.$$

These three terms are combined into the total loss

$$\mathcal{L} = \lambda_{\text{PDE}}\, \mathcal{L}_{\text{PDE}} + \lambda_{\text{IC}}\, \mathcal{L}_{\text{IC}} + \lambda_{\text{BC}}\, \mathcal{L}_{\text{BC}}.$$

The integers N_f, N_0, and N_b control how densely each constraint is sampled, and the scalar coefficients λ_{PDE}, λ_{IC}, and λ_{BC} balance their relative importance. The causal chain is: PDE, initial, and boundary errors contribute to $\mathcal{L}$; $\mathcal{L}$ depends on θ through T_θ and its derivatives; gradients of $\mathcal{L}$ update θ; and the updated θ changes the next forward pass, its derivatives, and the residual.

At this point the total loss is a scalar function of all weights and biases,

$$\theta = \{W^{[1]}, b^{[1]}, \ldots, W^{[N]}, b^{[N]}\},$$

so training computes the gradients

$$\frac{\partial \mathcal{L}}{\partial W^{[\ell]}}, \qquad \frac{\partial \mathcal{L}}{\partial b^{[\ell]}}$$

for every layer and updates the parameters, for example by gradient descent,

$$W^{[\ell]} \leftarrow W^{[\ell]} - \eta \frac{\partial \mathcal{L}}{\partial W^{[\ell]}}, \qquad b^{[\ell]} \leftarrow b^{[\ell]} - \eta \frac{\partial \mathcal{L}}{\partial b^{[\ell]}}.$$

The intermediate values $z^{[\ell]}$, $a^{[\ell]}$, T_θ, T_t, and T_{xx} are produced during each forward pass and are not directly updated by the loss; instead, changing $W^{[\ell]}$ and $b^{[\ell]}$ yields a new forward pass and therefore a new prediction $T_\theta(x,t)$. This is the mechanism by which the PDE, initial, and boundary constraints influence the learned solution.

Summary of the workflow

In brief, solving the 1D heat equation with a PINN proceeds as follows:

(i) Sample a space-time point (x, t) in the domain (interior, initial line, or boundaries depending on the loss term being evaluated).

(ii) Set the input activation

$$a^{[0]} = \begin{bmatrix} x \\ t \end{bmatrix}.$$

(iii) Propagate forward using

$$z^{[\ell]} = W^{[\ell]} a^{[\ell-1]} + b^{[\ell]}, \qquad a^{[\ell]} = \sigma\left(z^{[\ell]}\right),$$

and obtain $a^{[N]} = T_\theta(x, t)$.

(iv) Compute the partial derivatives $\partial T_\theta / \partial t$ and $\partial^2 T_\theta / \partial x^2$ via automatic differentiation.

(v) Form the PDE residual $R_\theta(x, t)$ by substituting these derivatives into the heat equation.

(vi) Aggregate $\mathcal{L}_{\text{PDE}}$, $\mathcal{L}_{\text{IC}}$, and $\mathcal{L}_{\text{BC}}$ over their respective point sets to obtain $\mathcal{L}$.

(vii) Back-propagate $\nabla_\theta \mathcal{L}$ and update $W^{[\ell]}$, $b^{[\ell]}$; repeat until $\mathcal{L}$ is acceptably small.

At the end of training, $T_\theta(x, t)$ is a continuous, differentiable approximation of the solution that satisfies the heat equation, initial condition, and boundary conditions simultaneously, without an explicit mesh-based spatial discretisation or conventional time-marching scheme.

Implementation logic in one pass

- Fix the PDE, initial, and boundary data and choose sampling strategies for interior and boundary points.
- For each batch of points, run a forward pass to get T_θ and use automatic differentiation to get T_t and T_{xx}.
- Compute R_θ at interior points and the IC/BC errors at their respective points.
- Form $\mathcal{L}$ as the weighted sum of these errors; $\mathcal{L}$ is a scalar function of θ only.
- Differentiate $\mathcal{L}$ with respect to all $W^{[\ell]}$ and $b^{[\ell]}$ and update them with the optimiser.
- Repeat until the loss and residuals reach acceptable levels.